PA2 Part 1: Executables

SKK UNIVERSITY

PA2

- **Goal**
  - The first goal is to implement 6 executables found in GNU Core Utilities (coreutil).
    - head
    - tail
    - cat
    - cp
    - mv
    - rm
  - Although the executables you will implement are simplified, most of the details are based on the man page of these executables, so you can refer to these if needed.
  - **Note: you should implement the behaviour of these executables (i.e., no execv, system(), etc.)**

PA2

- **Compilation**
  - These executables should be prefixed with pa2_
    - » (i.e. pa2_head, pa_tail, etc.).
  - The source files should be in pa2/executable_src while the executables should be in pa2/bin.
  - *make* should be able to compile and remove these executables.
  - pa2/
    - » **Makefile**
    - » **executable_src/**
    - » **shell_src/**
    - » bin/
    - » build/

PA2

- **Running commands**
  - If a command has [something], this means that the parameter is optional.
  - The file parameter is sometimes optional (i.e. [FILE]).
    - » If that is the case, the command must read standard input instead.
  - If argument has … , it means more than one argument can be passed.
  - You should add the **bin/** directory to PATH so that you can run the commands without specifying the exact path.
  - Note that other executables in PATH that you did not implement should still be runnable by the shell.

PA2

- **Running commands in shell**
  - If a command has [something], this means that the parameter is optional.
    - » When the file is optional (i.e., [FILE]), the command must read standard input instead.
  - If argument has ... , it means more than one argument can be passed.
  - Your shell should be able to run executables in PATH and pa2_* executables in the **bin/** directory without specifying the file path.

```
$ pa2_cat file.txt
$ cat file.txt
```

  - You should keep track of the exit status of when executing.
  - If the passed executable EXECUTABLE does not exist, print "EXECUTABLE: command not found" to standard error and set the status code as 127.

Hint

SKK UNIVERSITY

Parsing arguments

- **Extracting Arguments and Options**
  - You can use any function to extract arguments and options, even one from a third-party library as long as it compiles.
  - You have to specify C standard as `std=gnuXX` in Makefile and not `std=cXX`.
  - You don't have to consider about long options, `--help`, and other features.
  - getopt is used for the example due to its simplicity.

getopt

- `int getopt(int argc, char *argv[], const char *optstring);`
- `extern char *optarg;`
- `extern int optind, opterr, optopt;`
- **Goes through each option when called repeatedly (like strtok)**
  - You can use switch-case to check for the option.
  - *argc*, *argv* refer to the *argc*, *argv* in main()
    - » After calling *getopt*, *argv* would be changed so that the non-option arguments will be at the end.
  - *optstring* refers to the short names (letter) for the options (i.e. "abc:" for options –a –b –c VALUE)
    - » If ":" is after a letter, it means that the option requires an argument.
    - » If "+" is at the start, it means to stop after the first non-option.
  - *optind* is the index of the first non-option argument.
  - *optarg* refers to the argument passed to the option.

Iterating Through Options

- You can use switch-case to check for the option.
- If opt is ‘?’, it would print `argv[0]: invalid option – ‘...’` unless you set opterr to 0.

```
int opt;
opterr = 0; // prevent printing opt errors to stderr
while ((opt = getopt(argc, argv, "abc:h")) != -1) {
    switch (opt) {
        case 'a': a_flag = true; break;
        case 'b': b_flag = true; break;
        case 'c': c_val = strtoll(optarg, NULL, 10); break;
        case 'h': return EXIT_SUCCESS; // print help message in actual code.
        default: // actual value: (‘?’) -> unknown option
            fprintf(stderr, "Try '%s -h' for more information.\n", argv[0]);
            return EXIT_FAILURE;
    }
}
```

Example

```c
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
enum { BUFFER_SIZE = 8192 };

char to_escape_char(char c) {
    switch (c) {
        case 'n':
            return '\n';
        case 't':
            return '\t';
        case '\\':
            return '\\';
        default:
            return c;
    }
}
void write_when_needed(char *buffer, size_t *buffer_i) {
    if (*buffer_i >= BUFFER_SIZE - 1) {
        write(STDOUT_FILENO, buffer, *buffer_i);
        *buffer_i = 0;
    }
}
```

Example

```
int handle_options(int argc, char* argv[], bool* output_trailing_newline, bool*
enable_backslash_escaping) {
    opterr = 0; // prevent printing opt errors.
    int opt;
    while ((opt = getopt(argc, argv, "+neEh")) != -1) {
        switch (opt) {
            case 'n':
                *output_trailing_newline = false;
                break;
            case 'e':
                *enable_backslash_escaping = true;
                break;
            case 'E':
                *enable_backslash_escaping = false;
                break;
            case 'h':
                printf("Usage: %s [OPTION]... [STRING]...\n", argv[0]);
                printf("Echo the STRING(s) to standard output.\n");
                printf(" -n   do not output the trailing newline\n");
                printf(" -e   enable interpretation of backslash escapes\n");
                printf(" -E   disable interpretation of backslash escapes (default)\n");
                printf(" -h   display this help and exit\n");
                exit(0);
            case '?':
                return optind - 1; // treat unknown option as a non-option argument
        }
    }
    return optind; // optind is the index of the first non-option argument
}
```

Example

```
int main(int argc, char* argv[]) {
    bool output_trailing_newline = true;
    bool enable_backslash_escaping = false;
    int i = handle_options(argc, argv, &output_trailing_newline, &enable_backslash_escaping);

    char buffer[BUFFER_SIZE];
    size_t buffer_i = 0;


    for (; i < argc; i++) {
        for (size_t j = 0; argv[i][j] != '\0'; j++) {
            write_when_needed(buffer, &buffer_i);
            buffer[buffer_i++] = (enable_backslash_escaping && argv[i][j] == '\\' &&
                     argv[i][j + 1] != '\0') ? to_escape_char(argv[i][++j]) : argv[i][j];
        }

        if (i < argc - 1) {
            write_when_needed(buffer, &buffer_i);
            buffer[buffer_i++] = ' ';
        }
    }

    if (buffer_i > 0)            write(STDOUT_FILENO, buffer, buffer_i);
    if (output_trailing_newline) write(STDOUT_FILENO, "\n", 1);

    return 0;
}
```

Errors

SKK UNIVERSITY

Errors

- **Handling errors**
  - When an error occurs in the program you implement, you have to print a message to the standard error.
  - The actual message format would be mentioned on the slide for the command, but the format for most errors is “command: ERROR MESSAGE FROM STRERROR”.
    - » If you encounter any other error not found in the Error section of the command, print ***“command: %s”, where the string is the return value of strerror()***.
  - The message format is based on the error handling of the actual commands, so please refer to that as well if needed.
  - You can ignore edge cases that do not set errno if they are not specified in the command section (i.e., multiple arguments).

Programs

SKK UNIVERSITY

head

- **Synopsis**
  - pa2_head [OPTION] [FILE]
- **Description**
  - Prints the first 10 lines of a FILE to standard output.
  - If the file has fewer than 10 lines, it will print the whole file without any padding.
  - If no FILE is provided, or FILE is -, read standard input.
  - -n NUM prints up to NUM lines instead of 10
  - It is guaranteed that NUM is not negative.

head - errors

- **When FILE does not exist, print “pa2_head: cannot open 'FILE' for reading: No such file or directory”.**
  - Note that “No such file or directory” comes from errno.
  - `rm -f foo; pa2_head foo`
  - pa2_head: cannot open ‘foo' for reading: No such file or directory
- **When FILE is a directory, print “pa2_head: error reading ‘FILE’: Is a directory”.**
  - `mkdir bar; pa2_head bar ; rmdir bar`
  - pa2_head: error reading ‘bar’ : Is a directory
- **When NUM is not a number, print “pa2_head: invalid number of lines: ‘NUM’”**
  - `touch foo; pa2_head foo -n foo; rm foo`
  - pa2_head: invalid number of lines: ‘foo’

tail

- **Synopsis**
  - pa2_tail [OPTION] [FILE]
- **Description**
  - Prints the last 10 lines of a FILE to standard output.
  - If the file has fewer than 10 lines, it will print the whole file without any padding.
  - If no FILE is provided, or FILE is -, read standard input.
  - -n NUM prints up to NUM lines instead of 10
  - It is guaranteed that NUM is not negative.

tail - errors

- **When FILE does not exist, print “pa2_tail: cannot open 'FILE' for reading: No such file or directory”.**
  - Note that “No such file or directory” comes from errno.
  - `rm -f foo; pa2_tail foo`
  - pa2_tail: cannot open ‘foo' for reading: No such file or directory
- **When FILE is a directory, print “pa2_tail: error reading ‘FILE’: Is a directory”.**
  - `mkdir bar; pa2_tail bar; rmdir foo`
  - pa2_tail: error reading ‘bar’ : Is a directory
- **When NUM is not a number, print “pa2_tail: invalid number of lines: ‘NUM’”**
  - `touch foo; pa2_tail foo -n foo; rm foo`
  - pa2_tail: invalid number of lines: ‘foo’

cat

- **Synopsis**
  - pa2_cat [FILE]...
- **Description**
  - Concatenate FILE(s) to standard output.
  - Note for concatenation: Do not add a newline or space when concatenating. Just print the contents of the files as is.
  - You do not need to consider the maximum size of the files.
  - If no FILE is provided, or FILE is -, read standard input.

cat - errors

- **When a file does not exist, print “pa2_cat: FILE: No such file or directory”.**
  - `rm -f foo; pa2_cat foo`
  - pa2_tail: cannot open ‘foo' for reading: No such file or directory
- **When the file is a directory, print “pa2_cat: FILE: Is a directory”.**
  - `mkdir foo; pa2_cat foo; rmdir foo`
  - pa2_cat: foo: Is a directory
- **When you don’t have permissions to open a file, print “pa2_cat: FILE: Permission denied”.**
  - `touch foo; chmod u-r foo; pa2_cat foo`
  - pa2_cat: foo: Permission denied

cp

- **Synopsis**
  - pa2_cp SOURCE DEST
  - pa2_cp SOURCE... DIRECTORY
- **Description**
  - Copy SOURCE to DEST. If DEST exists and it is a file, overwrite DEST completely.
  - If it is an existing DIRECTORY, copy multiple SOURCE(s) to DIRECTORY.
  - Ignore cases when SOURCE is a directory. SOURCE is guaranteed to be a file

cp - errors

- **When no arguments are passed, print “pa2_cp: missing file operand”**
  - `pa2_cp`
  - pa2_cp: missing file operand
- **When only one argument is passed, print “pa2_cp: missing destination file operand after ‘SOURCE’”**
  - `pa2_cp foo`
  - pa2_cp: missing file operand after ‘foo’
- **When a file does not exist, print “pa2_cp: cannot stat ‘FILE’: No such file or directory”**
  - `rm -f foo && pa2_cp foo foo1`
  - pa2_cp: cannot stat ‘foo’: No such file or directory
- **When target directory does not exist, print “pa2_cp: cannot create regular file ‘DIRECTORY’: Not a directory”**
  - `touch foo && pa2_cp foo bar/`
  - pa2_cp: cannot create regular file ‘bar/’: Not a directory

cp - errors

- **When target directory is not a directory, print “pa2_cp: cannot stat ‘DIRECTORY’: Not a directory”**
  - `touch foo; touch bar; pa2_cp foo bar/; rm foo bar`
  - pa2_cp cannot stat ‘bar/’: Not a directory
- **When a directory cannot be accessed, print “pa2_cp: cannot create regular file ‘TARGET’: Permission denied”**
  - `mkdir bar; chmod u-w bar; touch foo; pa2_cp foo bar/; rm foo; rmdir bar`
  - pa2_cp: cannot create regular file ‘bar/foo’: Permission denied
- **When the SOURCE and TARGET are the same, print “pa2_cp: ‘SOURCE' and ‘TARGET’ are the same file”**
  - `touch foo; pa2_cp foo foo`
  - pa2_cp; ‘foo’ and ‘foo’ are the same file

mv - errors

- **When target directory is not a directory, print “pa2_mv: cannot stat ‘DIRECTORY’: Not a directory”**
  - `touch foo; touch bar; pa2_mv foo bar/; rm foo bar`
  - pa2_mv: cannot stat ‘bar/’: Not a directory
- **When a directory cannot be accessed, print “pa2_mv: cannot move ‘SOURCE’ to ‘TARGET’: Permission denied”**
  - `mkdir bar; chmod u-w bar; touch foo; pa2_mv foo bar/; rm foo; rmdir bar`
  - pa2_mv: cannot move ‘foo’ to ‘bar/foo’: Permission denied
- **When the SOURCE and TARGET are the same, print “pa2_mv: ‘SOURCE' and ‘TARGET’ are the same file”**
  - `touch foo; pa2_mv foo foo`
  - pa2_mv: ‘foo’ and ‘foo’ are the same file
- **When the DIRECTORY is a subdirectory of SOURCE, print “pa2_mv: cannot move ‘SOURCE’ to a subdirectory of itself, ‘TARGET’”**
  - `mkdir -p bar/foo; pa2_mv bar bar/foo; rm -r bar`
  - pa2_mv: cannot move ‘bar’ to a subdirectory of itself, ‘bar/foo/bar’

mv

- **Synopsis**
  - pa2_mv SOURCE DEST OR pa2_mv SOURCE… DIRECTORY
- **Description**
  - Rename SOURCE to DEST. If DEST exists and it is a file, overwrite DEST completely.
  - If it is an existing DIRECTORY, move multiple SOURCE(s) to DIRECTORY.
  - SOURCE can be a directory as well.
    - » When handling errors, print just the basename (‘foo.txt’ instead of ‘/…/…/foo.txt’ or ‘bar/foo.txt’)
  - TARGET: DEST or DIRECTORY/basename(SOURCE)

mv - errors

- **When no arguments are passed, print “pa2_mv: missing file operand”**
  - `pa2_mv`
  - pa2_mv: missing file operand
- **When only one argument is passed, print “pa2_mv: missing destination file operand after ‘SOURCE’”**
  - `pa2_mv foo`
  - pa2_mv: missing file operand after ‘foo’
- **When a file does not exist, print “pa2_mv: cannot stat ‘SOURCE’: No such file or directory”**
  - `rm -f foo; pa2_mv foo`
  - pa2_mv: cannot stat ‘foo’: No such file or directory
- **When target directory does not exist, print “pa2_mv: cannot move ‘SOURCE’ to ‘DIRECTORY’: Not a directory”**
  - `touch foo; pa2_mv foo bar/; rm foo`
  - pa2_mv: cannot move ‘foo’ to ‘bar/’: Not a directory

mv - errors

- **When target directory is not a directory, print “pa2_mv: cannot stat ‘DIRECTORY’: Not a directory”**
  - `touch foo; touch bar; pa2_mv foo bar/; rm foo bar`
  - pa2_mv: cannot stat ‘bar/’: Not a directory
- **When a directory cannot be accessed, print “pa2_mv: cannot move ‘SOURCE’ to ‘TARGET’: Permission denied”**
  - `mkdir bar; chmod u-w bar; touch foo; pa2_mv foo bar/; rm foo; rmdir bar`
  - pa2_mv: cannot move ‘foo’ to ‘bar/foo’: Permission denied
- **When the SOURCE and TARGET are the same, print “pa2_mv: ‘SOURCE' and ‘TARGET’ are the same file”**
  - `touch foo; pa2_mv foo foo`
  - pa2_mv: ‘foo’ and ‘foo’ are the same file
- **When the DIRECTORY is a subdirectory of SOURCE, print “pa2_mv: cannot move ‘SOURCE’ to a subdirectory of itself, ‘TARGET’”**
  - `mkdir -p bar/foo; pa2_mv bar bar/foo; rm -r bar`
  - pa2_mv: cannot move ‘bar’ to a subdirectory of itself, ‘bar/foo/bar’

mv - hint

- ```
  int rename(const char* oldpath,
             const char* newpath);
  ```
  - [stdio.h] Renames a file, moving the file from oldpath to newpath, overwriting any file at newpath.
  - Possible Errors:
    - » EACCES: Not enough write permissions for oldpath or the parent directory of oldpath or newpath; or not enough search (execute, x) permissions for one of the ancestors of oldpath/newpath.
    - » EISDIR: newpath is an existing directory, but oldpath is not a directory.
    - » EINVAL: newpath is a subdirectory of oldpath
    - » ENOENT: newpath, oldpath does not exist; some ancestor in newpath does not exist; newpath or oldpath is an empty string
    - » ENOTDIR: ancestor in newpath, oldpath is not a directory; oldpath is a directory, but newpath is not.
    - » ENOTEMPTY || EEXIST: newpath is a non-empty directory.
  ```

mv - hint

- `char* pa2_basename(const char* path);`
  - Gets the name of the file without the directory part (i.e. /home/spl/file -> file)
  - GNU Glibc implementation (https://sourceware.org/git/?p=glibc.git;a=blob;f=string/basename.c;h=dd2c8182dbc5df71b89100aab33ae98ff4e176ff;hb=refs/heads/release/2.42/master)

rm

- **Synopsis**
  - pa2_rm FILE...
- **Description**
  - Remove FILE(s)
  - If an errors occur for one file, the other files should still be removed.
  - No need to consider users not having sufficient permission to delete the file(s).

rm - errors

- **When no arguments are passed, print “pa2_rm: missing operand”**
  - `pa2_rm`
  - pa2_rm: missing operand
- **When a file does not exist, print “pa2_rm: cannot remove ‘FILE': No such file or directory”**
  - `pa2_rm foo`
  - pa2_rm: cannot remove ‘foo': No such file or directory
- **When a file is a directory, print “pa2_rm: cannot remove ‘FILE’: Is a directory”**
  - `mkdir foo; pa2_rm foo; rmdir foo`
  - pa2_rm: cannot remove ‘foo’: Is a directory

rm - hint

- `int unlink(const char *pathname);`
  - Deletes a name from the file system.
  - In unistd.h
  - Possible Errors:
    - » EACCES: Not enough write permissions for the parent of pathname or not enough search (execute) permissions for one of the ancestors of pathname
    - » EISDIR: pathname is a directory
    - » ENOENT: some ancestor in pathname does not exist; pathname is empty
    - » ENOTDIR: ancestor in pathname is not a directory;
    - » EPERM: Not enough permissions to unlink files/directorie